

Comparação de Algoritmos de Aprendizado de Máquina para Detecção de Malware Android: Um Estudo Empírico com o Dataset TUANDROMD

Fernando Pinéo de Abreu
Universidade de São Paulo
Nº USP: 17989742

Abstract

Este artigo apresenta um estudo empírico comparando quatro algoritmos de aprendizado de máquina supervisionado — Random Forest (RF), Máquina de Vetores de Suporte (SVM), K-Vizinhos Mais Próximos (KNN) e Árvore de Decisão (AD) — na tarefa de detecção de malware Android. Utilizamos o dataset TUANDROMD, composto por 4.464 aplicativos Android descritos por 241 features binárias (permissões e chamadas de API). Os experimentos foram conduzidos via validação cruzada estratificada de 10 folds sobre 90% dos dados, reservando 10% para exploração prévia. A comparação estatística foi realizada com o teste de Wilcoxon pareado com correção de Bonferroni. O Random Forest obteve o melhor desempenho ($F1 = 0,9969 \pm 0,0028$; $AUC = 0,9995 \pm 0,0006$), superando cada um dos demais algoritmos com diferença estatisticamente significativa após correção de Bonferroni ($p < 0,0083$). Os resultados sugerem que métodos ensemble são mais robustos para este problema, embora a alta prevalência de instâncias duplicadas no dataset imponha cautela na generalização das conclusões.

1 Introdução

A proliferação de aplicativos maliciosos para Android representa uma ameaça crescente à segurança de dispositivos móveis. Com mais de 3 bilhões de dispositivos ativos, a plataforma Android é alvo frequente de campanhas de malware que exploram permissões declaradas no manifesto dos aplicativos e chamadas de API do sistema operacional. A detecção automática por aprendizado de máquina tem se mostrado promissora nesse contexto, especialmente com base em análise estática (Wu et al., 2012).

Este trabalho investiga a seguinte questão de pesquisa inicial: *É possível detectar automaticamente malware Android a partir de features estáticas (permissões e chamadas de API) com alto*

desempenho? A partir da exploração dos dados, essa questão foi refinada para: *Qual dos quatro algoritmos — Random Forest, SVM, K-Vizinhos Mais Próximos e Árvore de Decisão — apresenta maior F1-score e AUC-ROC na detecção de malware Android, e as diferenças observadas são estatisticamente significativas sob validação cruzada estratificada de 10 folds?*

Utilizamos o dataset TUANDROMD (Mahindru and Sangal, 2021), composto por 4.464 instâncias rotuladas como *malware* (79,8%) ou *goodware* (20,2%), descritas por 241 features binárias. Os resultados indicam que o Random Forest supera os demais com significância estatística, embora as métricas elevadas ($F1 > 0,99$ para todos os algoritmos) sejam provavelmente infladas pela alta redundância de instâncias no dataset.

2 Dataset e Exploração

2.1 Descrição do Dataset

O TUANDROMD (Mahindru and Sangal, 2021) foi construído a partir de aplicativos Android coletados de repositórios públicos de APKs, incluindo amostras de malware de fontes catalogadas e aplicativos benignos de lojas oficiais. A coleta foi realizada por análise estática dos APKs, extraíndo automaticamente permissões declaradas no `AndroidManifest.xml` e chamadas de API identificadas no bytecode Dalvik (Mahindru and Sangal, 2021). Cada instância é descrita por 241 features binárias: 213 permissões Android e 28 chamadas de API do sistema. O protocolo exato de seleção dos aplicativos não é descrito em detalhes suficientes pelos autores para confirmar aleatoriedade estrita, caracterizando uma amostragem por conveniência dentro da moldura definida — limitação reconhecida na literatura de datasets públicos de segurança Android (Isaac and Tanimu, 2025).

2.2 Curadoria e Problemas Identificados

Durante a exploração identificamos os seguintes problemas.

Valores ausentes. Uma instância apresentava Label ausente e todas as suas 241 features simultaneamente ausentes, sendo removida por não ser imputável para aprendizado supervisionado. Nenhum outro valor ausente foi identificado nas demais colunas, confirmado programaticamente, resultando em 4.464 instâncias limpas.

Nome de coluna ofuscado. A coluna `getLastKnownLocation` foi renomeada para `getLastKnownLocation`, evidenciando que o processo de anonimização do dataset substituiu incorretamente a substring *known* por *goodware*.

Duplicatas. 3.802 instâncias compartilham vetores de features idênticos a pelo menos outra instância, resultando em apenas 662 vetores únicos no dataset. Uma mesma instância chega a se repetir até 792 vezes. Optamos por manter as duplicatas, pois com 241 variáveis binárias dois aplicativos distintos podem legitimamente compartilhar o mesmo vetor de features; além disso, a remoção reduziria o dataset de 4.464 para 662 instâncias (perda de 85,2%), comprometendo severamente a estabilidade da validação cruzada e a confiabilidade das estimativas de desempenho. Reconhecemos, contudo, que a manutenção das duplicatas pode permitir que vetores de treino apareçam no conjunto de teste dentro de um mesmo fold, constituindo potencial fonte de inflação das métricas.

Conflitos de rótulo. Dois vetores de features idênticos foram rotulados com classes opostas (*malware* e *goodware*), evidenciando ruído no processo de anotação.

Limitações e vieses. O protocolo de coleta não detalhado pelos autores impede confirmar representatividade estrita da moldura: aplicativos de repositórios de malware catalogados tendem a apresentar maior taxa de malware do que a distribuição real de uso em dispositivos. O desbalanceamento de classes (79,8% malware) reflete essa característica e constitui limitação para a generalização dos resultados.

2.3 Divisão Exploração / Execução

Seguindo a separação entre fases exploratória e confirmatória, 10% dos dados (446 instâncias, por amostragem estratificada com `random_state=42`)

foram reservados para análise exploratória. Os 90% restantes (4.018 instâncias) foram utilizados nos experimentos confirmatórios.

2.4 Execução da Exploração

A exploração da subamostra revelou padrões distintos de prevalência entre as classes. A Tabela 1 apresenta as cinco features com maior correlação absoluta com a variável-alvo, identificadas na fase exploratória (TP1, Seção 3.3). `openConnection` e `getLastKnownLocation` são mais prevalentes em *goodware* (correlação negativa), enquanto `RECEIVE_BOOT_COMPLETED` domina em *malware* — refletindo o comportamento típico de persistência no boot do dispositivo. `load` e `loadClass` também são mais prevalentes em *goodware*, indicando que aplicativos legítimos fazem uso explícito de carregamento dinâmico de bibliotecas.

Feature	Correlação	Interpretação Técnica
<code>openConnection</code>	-0,760	Goodware usa mais — ausência é indicador de malware
<code>getLastKnownLocation</code>	-0,711	Goodware usa mais — típico de spyware legítimo
<code>RECEIVE_BOOT_COMPLETED</code>	+0,694	Malware domina — execução persistente no boot
<code>load</code>	-0,557	Goodware usa mais — carregamento explícito de bibliotecas
<code>loadClass</code>	-0,541	Goodware usa mais — modularidade em apps legítimos

Table 1: Top-5 features por correlação absoluta com a variável-alvo na subamostra exploratória. Correlação negativa = associação com *goodware*; positiva = associação com *malware*.

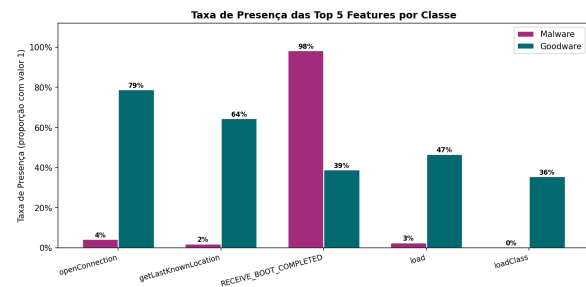


Figure 1: Taxa de presença (proporção de instâncias com valor 1) das cinco features mais discriminativas, estratificada por classe. Goodware apresenta maior prevalência nas APIs de conectividade e carregamento dinâmico, enquanto malware domina em `RECEIVE_BOOT_COMPLETED`.

2.5 Questão de Pesquisa Refinada

A exploração evidenciou dois fatores que subsidiam a questão refinada: (i) as features discriminadoras são chamadas de API binárias (presença/ausência), um espaço de alta dimensionalidade com correlações entre atributos; e (ii) o dataset é desbalanceado (79,8% malware). Esses fatores favorecem algoritmos robustos a correlações entre features e ao desbalanceamento de classes, motivando a comparação entre paradigmas distintos — ensemble, margem máxima, instâncias e simbólico — para identificar qual responde melhor ao problema. A questão refinada é: *Qual dos quatro algoritmos — Random Forest, SVM, KNN e Árvore de Decisão — apresenta maior F1-score e AUC-ROC na detecção de malware Android, e as diferenças observadas são estatisticamente significativas sob validação cruzada estratificada de 10 folds?*

A questão refinada incorpora ainda os achados da exploração: a ausência de APIs de conectividade (e.g., `openConnection`) e a presença de mecanismos de persistência (e.g., `RECEIVE_BOOT_COMPLETED`) emergem como os principais sinais discriminativos no TUANDROMD, motivando a escolha de algoritmos robustos a espaços de alta dimensionalidade com features binárias esparsas.

Cabe, contudo, reconhecer as limitações do mapeamento entre variáveis e fatores. O Fator 1 (APIs de conectividade) é representado por variáveis binárias que indicam presença ou ausência de chamada, sem distinguir se a conexão estabelecida é maliciosa ou legítima — dois aplicativos com comportamentos opostos recebem o mesmo valor 1 para `openConnection`. O Fator 2 (execução dinâmica e persistência) enfrenta limitação similar: `loadClass=1` captura carregamento de classes, mas não diferencia bibliotecas legítimas de código malicioso injetado dinamicamente. O Fator 3 (permissões de dados sensíveis) sofre de um problema estrutural do próprio dataset: permissões declaradas no `AndroidManifest.xml` não garantem uso efetivo — um aplicativo pode declarar `ACCESS_FINE_LOCATION` sem nunca acessar a localização do usuário, reduzindo o poder discriminativo real da variável. Essas limitações são inerentes à abordagem de análise estática e constituem motivação para estudos futuros com análise dinâmica complementar.

3 Materiais e Métodos

3.1 Base de Dados e Divisão

A partição dos dados foi definida antes de qualquer experimento, em duas etapas exclusivas e permanentes. Os 446 exemplos reservados para a fase exploratória (10%, estratificados com `random_state=42`) funcionam como partição de isolamento permanente: não participaram em nenhuma etapa dos experimentos confirmatórios, equivalendo a um conjunto de teste externo mantido isolado desde o início do estudo. Os 4.018 exemplos restantes (90%) constituem a partição de execução, sobre a qual foram conduzidos todos os experimentos.

Esses 4.018 exemplos foram submetidos a validação cruzada estratificada de 10 folds (`StratifiedKFold`, `shuffle=True`, `random_state=42`), gerando em cada iteração um conjunto de treino (≈ 3.616 instâncias) e um conjunto de validação (≈ 402 instâncias), preservando a proporção de classes em ambos.

3.2 Pré-processamento

Para SVM e KNN, aplicou-se normalização Min-Max (`MinMaxScaler`) exclusivamente sobre os dados de treino de cada fold, com posterior transformação do conjunto de teste — eliminando qualquer vazamento de informação. Random Forest e Árvore de Decisão são invariantes à escala e não requerem normalização.

3.3 Técnicas de Aprendizado

Selecionamos quatro algoritmos que representam paradigmas distintos de aprendizado supervisionado. O **Random Forest** (Breiman, 2001) é um método ensemble baseado em bagging de árvores de decisão, robusto a correlações entre features e a overfitting. O **SVM** (Cortes and Vapnik, 1995) é um classificador de margem máxima eficaz em espaços de alta dimensionalidade. O **KNN** é um método baseado em instâncias, sem fase de treino explícita, útil como baseline de complexidade baixa. A **Árvore de Decisão** serve como referência simbólica interpretável. Essa diversidade de paradigmas permite avaliar se abordagens fundamentalmente diferentes respondem de forma equivalente ao problema.

3.4 Hiperparâmetros e Sementes

Os quatro algoritmos foram configurados conforme a Tabela 2. Todos os hiperparâmetros não listados

mantiveram os valores padrão do scikit-learn 1.6.1.

Algoritmo	Hiperparâmetros
Random Forest	n_estimators=100, criterion=gini max_features=sqrt, bootstrap=True
SVM	C=1.0, kernel=rbf gamma=scale, probability=True
KNN	n_neighbors=5, weights=uniform metric=minkowski ($p = 2$)
Árvore (AD)	criterion=gini, max_depth=None

Table 2: Hiperparâmetros dos algoritmos. random_state=42 em todos os modelos estocásticos e no StratifiedKFold.

3.5 Métricas de Avaliação

O **F1-score binário** (pos_label = malware) foi adotado como métrica principal por penalizar tanto falsos positivos (goodware classificado como malware) quanto falsos negativos (malware não detectado), sendo adequado ao desbalanceamento da classe. O **AUC-ROC** complementa a avaliação ao medir a discriminabilidade independente do limiar de decisão, capturando a qualidade probabilística dos modelos.

3.6 Comparação Estatística

Para cada par de algoritmos, aplicamos o **teste de Wilcoxon pareado** sobre os 10 F1-scores dos folds (scipy.stats.wilcoxon). Como foram realizadas 6 comparações, aplicou-se correção de Bonferroni: $\alpha_{corrigido} = 0,05/6 = 0,0083$.

3.7 Reprodutibilidade

Os experimentos foram executados em Google Colab (Python 3.12.13, Linux 6.6.113+, CPU, sem GPU). Bibliotecas utilizadas: pandas 2.2.2 (manipulação de dados), numpy 2.0.2 (operações numéricas), scikit-learn 1.6.1 (modelos, validação, métricas), scipy 1.16.3 (teste de Wilcoxon), matplotlib 3.10.0 e seaborn 0.13.2 (visualizações). O código-fonte completo está disponível no material suplementar, com todas as sementes fixadas em random_state=42.

4 Resultados

4.1 Ambiente e Tempo de Execução

O tempo médio de treino por fold variou de 0,010 s (KNN, *lazy learner* sem fase de treino explícita) a 1,633 s (SVM, $\sigma = 1,003$ s). O Random Forest levou em média 0,724 s por fold ($\sigma = 0,550$ s);

a Árvore de Decisão, 0,038 s ($\sigma = 0,013$ s). O alto desvio do SVM e do Random Forest reflete a variação na composição dos folds, esperada em datasets com alta proporção de duplicatas.

4.2 Desempenho dos Algoritmos

A Tabela 3 apresenta os resultados consolidados nos 10 folds. O Random Forest obteve o maior F1 médio (0,9969) e o maior AUC médio (0,9995), além do menor desvio padrão no AUC, indicando maior estabilidade. A Árvore de Decisão, apesar de F1 competitivo, apresenta AUC substancialmente inferior ($0,9819 \pm 0,0127$), sugerindo menor qualidade probabilística.

Algoritmo	F1	σ	AUC	σ
Random Forest	0,9969	0,0028	0,9995	0,0006
SVM	0,9925	0,0034	0,9989	0,0013
KNN	0,9904	0,0028	0,9929	0,0061
Árvore (AD)	0,9927	0,0032	0,9819	0,0127

Table 3: F1-score e AUC-ROC médios (desvio padrão) na validação cruzada estratificada de 10 folds. Melhor valor em negrito.

A Figura 2 exibe a distribuição dos F1-scores por fold para cada algoritmo.

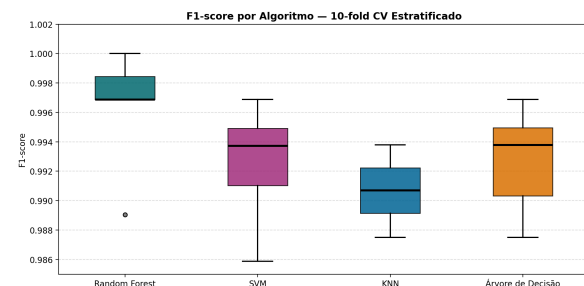


Figure 2: Distribuição dos F1-scores por fold para cada algoritmo na validação cruzada estratificada de 10 folds.

O Random Forest apresentou um outlier no Fold 6 (F1=0,9890), único fold onde o desempenho destoou dos demais (média dos outros 9 folds: F1=0,9976). Uma possível explicação é a concentração de instâncias de fronteira nesse fold — como os dois vetores com conflito de rótulo identificados na exploração —, embora não seja possível confirmar essa hipótese sem análise direta da composição do fold.

A Figura 3 apresenta os F1-score e AUC-ROC médios com barras de erro para cada algoritmo.

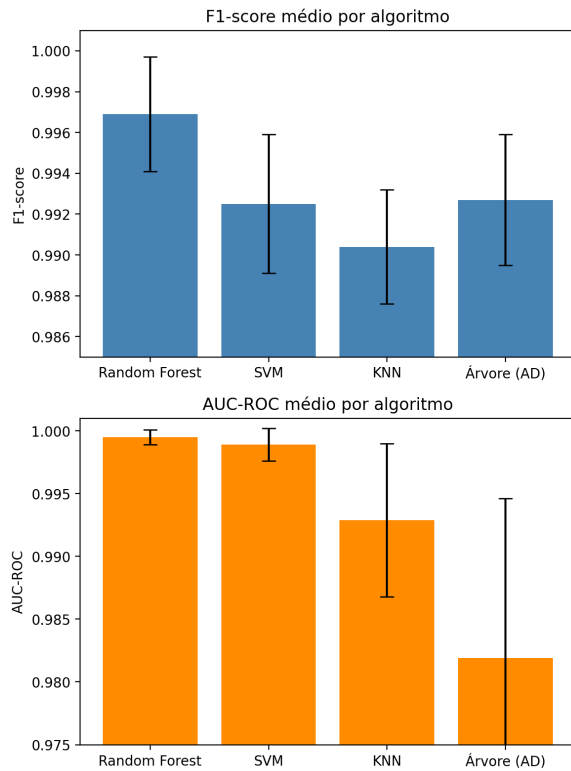


Figure 3: Acima: F1-score médio por algoritmo. Abaixo: AUC-ROC médio por algoritmo. Barras de erro indicam desvio padrão nos 10 folds.

4.3 Comparação Estatística

A Tabela 4 apresenta os resultados do teste de Wilcoxon pareado. O Random Forest superou todos os demais com $p \leq 0,0039$, abaixo do limiar corrigido. Não houve evidência suficiente para afirmar diferença entre SVM e AD ($p = 0,6953$), nem entre KNN e AD ($p = 0,0508$). A Figura 4 visualiza os p-valores entre todos os pares.

Par	p-valor	Sig. (Bonf.)
RF vs SVM	0,0020	✓
RF vs KNN	0,0020	✓
RF vs AD	0,0039	✓
SVM vs KNN	0,0078	✓
SVM vs AD	0,6953	×
KNN vs AD	0,0508	×

Table 4: Teste de Wilcoxon pareado sobre os F1-scores dos 10 folds. $\alpha_{Bonf.} = 0,0083$. ✓ = diferença significativa.

5 Discussão

O Random Forest apresentou o melhor desempenho global e a maior robustez estatística, superando os demais em todos os pares com sig-

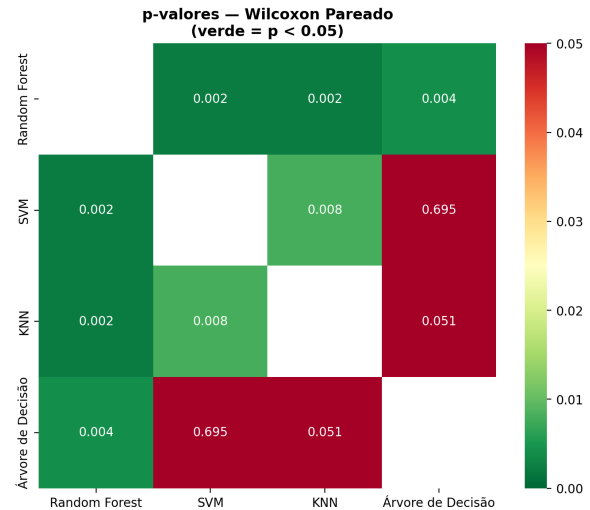


Figure 4: Mapa de calor dos p-valores do teste de Wilcoxon pareado. Verde indica diferença estatisticamente significativa após correção de Bonferroni ($p < 0,0083$).

nificância após correção de Bonferroni. Esse resultado é coerente com a natureza do problema: features binárias em alta dimensionalidade com correlações entre atributos favorecem métodos ensemble que combinam múltiplos classificadores com seleção aleatória de features.

SVM, KNN e Árvore de Decisão não apresentam diferenças estatisticamente sustentadas entre si (para os pares SVM vs. AD e KNN vs. AD). O par SVM vs. KNN atingiu $p = 0,0078$ — significativo sem Bonferroni, mas não após a correção — ilustrando a importância de ajustar o nível de significância para múltiplas comparações.

Os resultados devem ser interpretados com cautela. O dataset contém 3.802 instâncias duplicadas (com até 792 cópias de uma mesma instância), o que pode permitir que versões de instâncias de treino apareçam no conjunto de teste do mesmo fold, inflando artificialmente as métricas. Além disso, os hiperparâmetros foram mantidos em seus valores padrão, sem otimização.

Como direções futuras, sugere-se: (i) repetir os experimentos após remoção das duplicatas para avaliar o impacto real da redundância; (ii) aplicar ajuste fino de hiperparâmetros via busca em grade com validação cruzada aninhada; (iii) investigar técnicas de balanceamento (e.g., SMOTE) para mitigar o desbalanceamento de classes; e (iv) validar os resultados em datasets independentes de malware Android.

Limitations

As principais limitações deste trabalho são: (i) a alta taxa de duplicatas no TUANDROMD provavelmente infla as métricas reportadas, e não é possível afirmar que $F1 > 0,99$ se manteria em dados sem redundância; (ii) os hiperparâmetros não foram otimizados, limitando a comparação a configurações padrão; (iii) o estudo restringe-se a um único dataset, comprometendo a generalização; e (iv) o estudo restringe-se a features estáticas extraídas em tempo de análise, não capturando comportamento dinâmico em execução, o que pode limitar a detecção de malware polimórfico avançado.

Ethics Statement

Este trabalho utiliza um dataset público de aplicativos Android para fins de pesquisa em segurança computacional. Nenhum dado pessoal de usuários foi coletado ou processado. Os modelos desenvolvidos têm potencial de aplicação legítima na proteção de dispositivos móveis.

References

- Leo Breiman. 2001. [Random forests](#). *Machine Learning*, 45(1):5–32.
- Corinna Cortes and Vladimir Vapnik. 1995. [Support-vector networks](#). *Machine Learning*, 20(3):273–297.
- J. Isaac and J. Tanimu. 2025. [Android malware detection using random forest algorithm](#). Zenodo. Accessed: 2026-04-05.
- Arvind Mahindru and A. L. Sangal. 2021. [TUANDROMD \(tuandroid malware dataset\)](#). Mendeley Data, v1, Accessed: 2026-04-05.
- Dong-Jie Wu, Ching-Hao Mao, Te-En Wei, Hahn-Ming Lee, and Kuo-Ping Wu. 2012. DroidMat: Android malware detection through manifest and api calls tracing. In *Proceedings of the 7th Asia Joint Conference on Information Security (AsiaJCIS)*, pages 62–69. IEEE.